Performance testing with Apache JMeter: interview questions, answered with this project

Questions about performance testing and JMeter that come up in SDET and QA lead interviews. Where the project shows the idea, the answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/jmeter-performance-testing.

Versions: Apache JMeter 5.6.3, jmeter-maven-plugin 3.8, Java 21.

Contents

1. Performance testing basics (Q1-Q14)
2. JMeter building blocks (Q15-Q34)
3. Running and reading results (Q35-Q48)
4. Process and scenarios (Q49-Q60)

1. Performance testing basics

Q1. What are the main types of performance test?

Smoke: a tiny load, to prove the script works. Load: expected traffic. Stress: rising load, to find the limit. Spike: a sudden burst, to see the effect and the recovery. Soak: normal load for hours, to find leaks.

In the project: the `smoke`, `load`, `stress` and `spike` Maven profiles. Soak isn't included.

Q2. What is the difference between throughput and response time?

Throughput is work per second. Response time is how long one request takes. They're linked: past a limit, throughput stays flat and response time keeps rising.

In the project: the stress result: flat at about 66 bookings per second while p95 climbs to 1.3 s.

Q3. Why use percentiles and not the average?

An average hides the slow tail. If 5 % of users wait 10 seconds, the average can still look fine. p95 says what 95 % of requests stayed within.

In the project: the gate uses p95.

Q4. What is an SLO?

A service level objective: a measurable target, such as "95 % of bookings within 300 ms, no errors". It turns a performance test from a report into a pass or fail.

In the project: `config/slo/<profile>.properties`.

Q5. What is Little's Law?

Users in the system = arrival rate x time in the system. It's a quick sanity check on a load model. It also explains why adding users past a limit only adds waiting.

In the project: with a 66 per second limit, 120 users in a closed loop must each wait longer, which is the p95 rise.

Q6. What is a bottleneck, and how do you find one?

The resource that limits the whole system. Raise the load, find where throughput stops rising, and then look at what is saturated at that moment: CPU, a pool, a lock, a database.

In the project: four payment connections held for 40-80 ms: 4 / 0.06 s = about 66 per second, which is what the stress test measured.

Q7. What is the knee of the curve?

The point where response time starts to climb steeply as load rises. It's a better capacity figure than the point of failure, because users suffer long before errors appear.

In the project: about 54 users.

Q8. Open or closed workload model?

Closed: a fixed number of users, each waiting for an answer before the next step, so a slow system receives less load. Open: arrivals continue regardless. Real public traffic is closer to open. JMeter thread groups are closed,

and a closed model can hide how bad an overload really is.

In the project: a closed model, stated as a limit in `docs/framework-design.md`. My k6 project has open-model tests.

Q9. What is think time, and why does it matter?

The pause a real user makes between actions. Without it, each thread acts like many real users, so the load model is wrong.

In the project: 200-400 ms, twice per round.

Q10. How do you decide how many users to simulate?

From production data: peak requests per second, session length, and the mix of actions. With no data, agree an assumption with the business and write it down.

Q11. Why does a load test need a realistic mix of actions?

Because different actions stress different resources. Testing only the home page says little about checkout.

In the project: one flow: login, browse, book, read back.

Q12. What does a test environment need for useful results?

It should resemble production in size and data. Nothing else should use it during the test, and the load generator must not be the bottleneck.

Q13. Why not run heavy load from one laptop?

The generator's own CPU, memory or network can become the limit, so you measure yourself. Watch the generator too, and spread large tests across machines.

Q14. Is a fast result enough?

No. A system can be fast and wrong. Check correctness under load too: lost updates, double bookings, wrong totals.

In the project: `Check no overselling`.

2. JMeter building blocks

Q15. What is a Thread Group?

The virtual users: how many, how quickly they start, and how long they run.

In the project: `Base load` and `Burst`.

Q16. What do setUp and tearDown thread groups do?

They run before and after the normal groups, for preparing data and for final checks.

In the project: `After the test`, a tearDown group, checks the seat count once all load has stopped.

Q17. What is a Sampler?

The thing that sends a request. The HTTP Request sampler is the usual one.

Q18. What is the order of execution in JMeter?

For each sampler: pre-processors, timers, the sampler, post-processors, assertions, listeners. The order in the tree matters less than people expect, because the element type decides much of it.

Q19. How do scoping rules work?

An element applies to the samplers in its branch of the tree. A config element at the top applies to everything, and one under a sampler applies only to it.

In the project: the token header sits under the booking requests only.

Q20. How does a timer's scope catch people out?

A timer applies to every sampler in its scope. Two timers in a thread group delay each request by both. Put a timer under a Flow Control Action to pause once.

In the project: this really happened. 20 users made 8 bookings per second where about 26 were expected, and the per-round time gave it away.

Q21. What does a Transaction Controller do?

It groups requests into one measured business step. With "generate parent sample" its result is one row per transaction.

In the project: `Login`, `Browse events`, `Book seats`, `Check booking`.

Q22. What is correlation?

Taking a dynamic value from one response and sending it in a later request, such as a token or an id.

In the project: JSON Extractors keep `token`, `eventId` and `bookingId`.

Q23. JSON Extractor or Regular Expression Extractor?

Use the JSON Extractor for JSON: it's clearer and doesn't break on formatting. Use a regular expression only when no structured option exists.

Q24. How do you pick a random value from a response?

Set the extractor's match number to 0.

In the project: `Pick an event`, so load spreads over all 20 events.

Q25. How do you feed test data?

A CSV Data Set Config: variables per column, with sharing and recycling settings.

In the project: `users.csv`, 200 users, shared by all threads.

Q26. What does "Sharing mode" mean in CSV Data Set?

Whether all threads read one shared position in the file, each thread group has its own, or each thread does.

Q27. What is HTTP Request Defaults?

Shared settings for all requests: host, port, timeouts. Change the environment in one place.

In the project: `API address`, with the port from a property.

Q28. What is `__P`?

A function that reads a JMeter property with a default, such as `${__P(users,5)}`. It lets one plan serve many load shapes.

In the project: every load setting.

Q29. Variables or properties?

Variables belong to one thread. Properties are shared by all threads. Use properties for settings and shared counters, with care about concurrent updates.

In the project: `limitedSeatsBooked` is a property, updated in a `synchronized` block.

Q30. Why does an assertion accept a 409?

Because a correct API answers 409 for a sold-out event. "Ignore status" or `assume_success` lets you say which failure codes are correct behaviour, instead of counting them as errors.

In the project: `Status 201 or 409`.

Q31. Why prefer Groovy with JSR223?

It compiles and caches scripts, so it scales. BeanShell is much slower under load. Set "cache compiled script" and use variables through `vars`, not `${...}` inside the script.

In the project: `Count confirmed seats` and `Sold matches confirmed bookings`.

Q32. What does an If Controller's "interpret as variable expression" do?

It evaluates a function or variable that gives true or false, without a slower JavaScript evaluation.

In the project: `Only if booked`.

Q33. What is a Once Only Controller?

It runs its children on each thread's first loop only.

In the project: login, once per virtual user.

Q34. Which timers are worth knowing?

Constant and Uniform Random for think time, Constant Throughput to hold a rate, and Synchronizing Timer to release threads together.

3. Running and reading results

Q35. Why run in non-GUI mode?

The GUI is for building and debugging. It uses a lot of memory and distorts results under load.

In the project: the Maven plugin always runs non-GUI.

Q36. Why avoid listeners during a test?

View Results Tree and similar listeners keep data in memory and slow the generator. Write a results file and report afterwards.

In the project: the plan has no listeners.

Q37. What is a JTL file?

JMeter's results file, in CSV or XML. CSV is smaller and faster.

In the project: `SloGate` reads it, finding columns by name.

Q38. What is the HTML dashboard?

A report generated from the JTL: percentiles, throughput over time, errors, response time graphs.

In the project: `target/jmeter/reports/booking/index.html`, kept as a CI artifact.

Q39. Does JMeter have pass or fail thresholds?

Not for percentiles. Assertions fail single samples, but nothing fails a run for a slow p95. You add a gate that reads the results, or use a plugin.

In the project: `SloGate`.

Q40. How do you run JMeter in CI?

The Maven plugin or the command line in a job, then a gate on the results, with the report kept as an artifact. Start with a small smoke run and keep the long tests off the pull request path.

In the project: `.github/workflows/perf.yml`.

Q41. How can a performance gate pass falsely?

By reading old results, by a renamed transaction vanishing from the check, or by a run that's too short.

In the project: this really happened: `-DskipTests` also skipped JMeter, so the gate read the previous run and passed. Now the plugin ignores that flag, a label with no samples is a miss, and each SLO sets a minimum sample count.

Q42. Are CI runners good for absolute numbers?

No. Shared runners vary. Use wide limits to catch big regressions, and do precise measurement on dedicated machines.

In the project: SLOs leave room over the local figures.

Q43. How do you read a stress test?

Plot throughput and p95 against users. Find where throughput stops rising, and see whether errors appear and whether the system recovers.

In the project: `docs/images/stress-result.png`.

Q44. What is a good sign in a spike test?

The system slows but doesn't fail, and returns to normal quickly after the burst.

In the project: p95 about 860 ms in the burst, back to about 87 ms within one 10-second window, no errors.

Q45. Should you warm up before measuring?

Usually yes. JIT compilation and cold caches make the first minute unrepresentative. Use a ramp-up, and exclude or at least recognise that period.

Q46. Why is ramp-up important?

All users starting at once is a spike, not a load test. It also hides the point where the system begins to struggle.

Q47. How do you scale JMeter?

Distributed mode, with one controller and several workers, or independent generators with results merged afterwards. Tune the JVM heap and keep the plan lean.

Q48. What would you monitor on the server during a test?

CPU, memory, garbage collection, thread and connection pools, database time, and queues. Client numbers show the symptom; server numbers show the cause.

4. Process and scenarios

Q49. JMeter or k6?

JMeter has a GUI, many protocols and wide enterprise use. k6 is code-first, light on resources, and has open-model executors and built-in thresholds. I choose by team and protocol needs, and I've built a project in each.

Q50. JMeter or Gatling?

Gatling is code-first and efficient, with good reports. JMeter is more widely known and has more protocol support.

Q51. How do you version `.jmx` files?

Keep them in Git, keep settings in properties, and keep plans small. They're XML, so diffs are noisy: review changes in the GUI as well.

Q52. Response time doubled after a release. What do you do?

Confirm it on a repeat run with the same data. Compare the two runs by transaction, find which step changed, check server metrics, and look at what changed in the release.

Q53. Errors appear at 200 users. What next?

Read the error types first. Timeouts, 5xx and connection refusals point to different causes. Then correlate with server metrics at that moment, and confirm the generator wasn't the limit.

Q54. Your load test passes, but production is slow. Why?

Usually the model was wrong: different data volumes, a different action mix, no cache misses, or a smaller environment.

Q55. How do you test a login-heavy system when users are limited?

Reuse users with the CSV set to recycle, log in once per user, and check whether the system blocks parallel sessions.

In the project: 200 users, recycled.

Q56. How do you prove a performance test can catch a problem?

Break the system on purpose and check the gate fails.

In the project: a pool of one, and selling one seat too many. See section 9 of `docs/framework-design.md`.

Q57. What should a performance test report say?

The goal, the load model, the environment, the results against the SLOs, the bottleneck, and a recommendation. Lead with the answer, not the graphs.

Q58. When should performance testing happen in a project?

Early and often: a small smoke in CI on every change, load tests regularly, and the big tests before major releases.

Q59. How do you avoid hitting someone else's server?

Only load systems you own or have written permission to test. A load test on a third party can look like an attack.

In the project: each run starts its own copy of the API.

Q60. How would you explain this to a manager?

We know the booking system handles about 66 bookings per second, that it slows but doesn't fail above that, that it recovers from a burst in seconds, and that it never sells a seat twice. If a change breaks any of those, the build fails.